LAPORAN UTS MOBILE PROGRAMMING
(APLIKASI ANILIST MINI)



Nama : Erai Bagusalim
NIM : 240605110088
Kelas : A

JURUSAN TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG
2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang dan Gambaran Aplikasi

Anime adalah media hiburan animasi asal Jepang yang sangat digemari dan ditonton oleh jutaan pengguna setiap harinya. Seiring meluasnya penggunaan telepon pintar, semakin banyak orang yang membaca referensi, mengecek skor rating, dan mencari judul anime langsung dari perangkat genggam mereka. Meskipun demikian, mencari anime tertentu di antara ribuan judul, mengetahui skor penilaian, jumlah episodenya, format tayangannya, serta membaca sinopsis ringkasnya masih memerlukan beberapa langkah jika dilakukan melalui penelusuran peramban yang tampilannya kurang ringkas pada layar ponsel.

Berdasarkan kebutuhan tersebut, dikembangkan aplikasi AniList Mini, yaitu aplikasi mobile berbasis Flutter yang menyajikan katalog anime dan rincian informasinya dalam tampilan yang modern, sederhana, dan mudah dicari. Seluruh data diambil secara dinamis dari REST API publik (Tenrai API v1 yang kompatibel dengan skema Jikan API v4), sehingga aplikasi tidak menyimpan data anime secara tetap di dalam kode program.

Tujuan pengembangan aplikasi ini adalah sebagai berikut.

1. Membangun aplikasi mobile menggunakan Flutter dan bahasa Dart yang menyajikan data dari API.
2. Menerapkan pengambilan data melalui permintaan HTTP dan mengonversi respons JSON menjadi objek Dart melalui class model AnimeModel.
3. Menerapkan asynchronous programming menggunakan Future dan FutureBuilder dengan penanganan kondisi loading, success, dan error.
4. Menghasilkan aplikasi yang bermanfaat bagi pengguna yang ingin mencari dan mengeksplorasi katalog anime beserta informasinya.

Sasaran pengguna aplikasi ini adalah pelajar, mahasiswa, dan masyarakat umum berbahasa Indonesia yang ingin mencari anime, memahami rating dan informasi singkatnya, serta membaca sinopsis cerita melalui telepon pintar.

Secara umum, aplikasi terdiri atas dua halaman. Halaman pertama menampilkan daftar anime terpopuler yang dapat dicari. Halaman kedua menampilkan detail satu anime terpilih, berisi poster besar, rating, tipe, episode, tahun, status, daftar genre, dan sinopsis lengkap. Kedua halaman mengambil datanya dari API dan menampilkan indikator yang sesuai saat data sedang dimuat atau gagal dimuat.

1.2 Fitur Aplikasi

Fitur-fitur utama yang tersedia pada aplikasi:

No	Fitur	Fungsi	Manfaat bagi Pengguna
1	Daftar Top Anime	Menampilkan 24 anime terpopuler berisi poster, judul, rating, dan tipe.	Pengguna melihat seluruh anime populer dalam satu daftar kisi yang teratur.
2	Pencarian anime	Menyaring daftar anime dari server berdasarkan kata kunci judul.	Anime yang dicari dapat ditemukan dengan cepat dan akurat.
3	Detail anime	Menampilkan rincian lengkap: poster, rating, tipe, episode, tahun, status, genre, sinopsis.	Pengguna dapat membaca dan memahami latar belakang anime sebelum menonton.
4	Refresh data	Memuat ulang katalog Top Anime dan membersihkan kata kunci pencarian.	Pengguna dapat memperbarui daftar katalog kapan saja secara instan.
5	Indikator loading	Menampilkan indikator saat data sedang diambil dari server.	Pengguna tahu bahwa aplikasi sedang bekerja mengambil data.
6	Penanganan error dan coba lagi	Menampilkan pesan kesalahan yang jelas dan tombol untuk mengulangi permintaan.	Pengguna tidak melihat layar kosong ketika koneksi bermasalah.

BAB 2. PEMBAHASAN

2.1 Persiapan Project dan Package

a. Pembuatan project Flutter

Project dibuat menggunakan Android Studio / Flutter CLI dengan memilih jenis project Application dan memberi nama anilist_mini. Aplikasi dijalankan dan diuji pada emulator Android Pixel 5 yang tersedia di Android Studio serta pada Google Chrome dengan tampilan mobile.

b. Struktur folder dan berkas utama

Folder / Berkas	Fungsi
lib/main.dart	Titik masuk aplikasi; mengatur tema Material 3 Dark Mode dan halaman awal.
lib/models/anime_model.dart	Class model AnimeModel untuk data pada katalog dan detail anime.
lib/services/anime_service.dart	Mengirim permintaan HTTP, memeriksa status respons, mengubah JSON, dan menangani kesalahan.
lib/pages/home_page.dart	Halaman daftar anime dengan pencarian dan FutureBuilder.
lib/pages/detail_page.dart	Halaman detail anime dengan FutureBuilder untuk rincian metadata lengkap.
test/anime_model_test.dart	Unit test untuk memeriksa konversi JSON ke model dan penanganan nilai null.
test/anime_service_test.dart	Unit test untuk service: endpoint, query encoding, status 429/500, dan jaringan.
test/widget_test.dart	Widget test untuk memverifikasi penampilan kartu anime dan kondisi hasil kosong.
pubspec.yaml	Daftar dependensi dan konfigurasi project.

c. Package yang digunakan

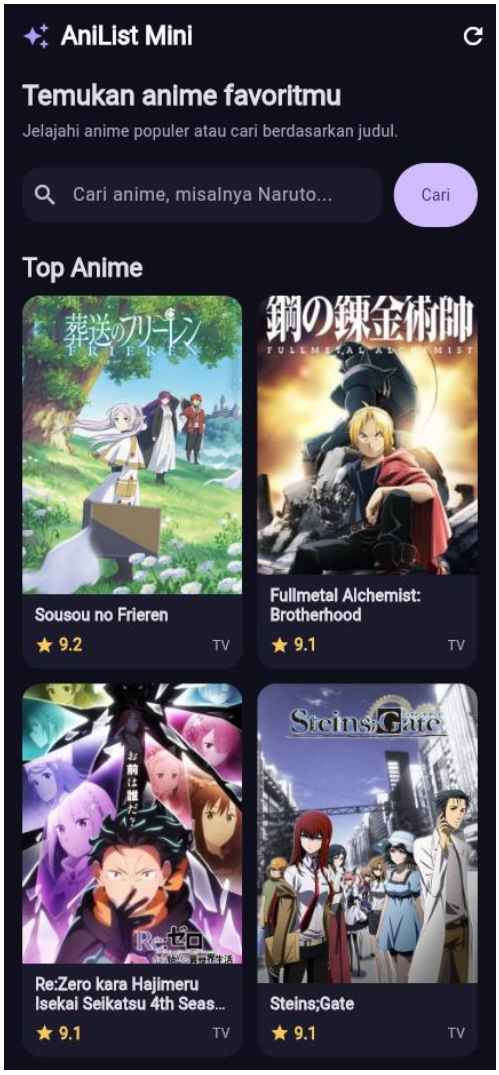
Package	Fungsi	Alasan Penggunaan
http	Mengirim permintaan HTTP GET ke API.	Package resmi yang sederhana dan direkomendasikan dokumentasi Flutter untuk pengambilan data. Juga menyediakan MockClient untuk pengujian tanpa internet.
flutter_lints	Aturan analisis kode (lint).	Menjaga gaya penulisan kode tetap konsisten dan mendeteksi kesalahan umum.
flutter_test	Unit test dan widget test.	Memeriksa fungsi aplikasi secara sistematis dan otomatis.

2.2 Desain Antarmuka Pengguna (User Interface)

Antarmuka dirancang elegan dengan tema Material 3 Dark Mode (latar #10101C dan kontainer kartu #1C1B2B). Aplikasi terdiri atas dua halaman, yaitu halaman daftar anime dan halaman detail anime. Komponen yang digunakan antara lain Scaffold dan AppBar, TextField untuk pencarian, GridView.builder untuk daftar katalog anime yang panjang, Card dan InkWell untuk setiap anime, serta Chip dan Wrap untuk daftar genre anime.

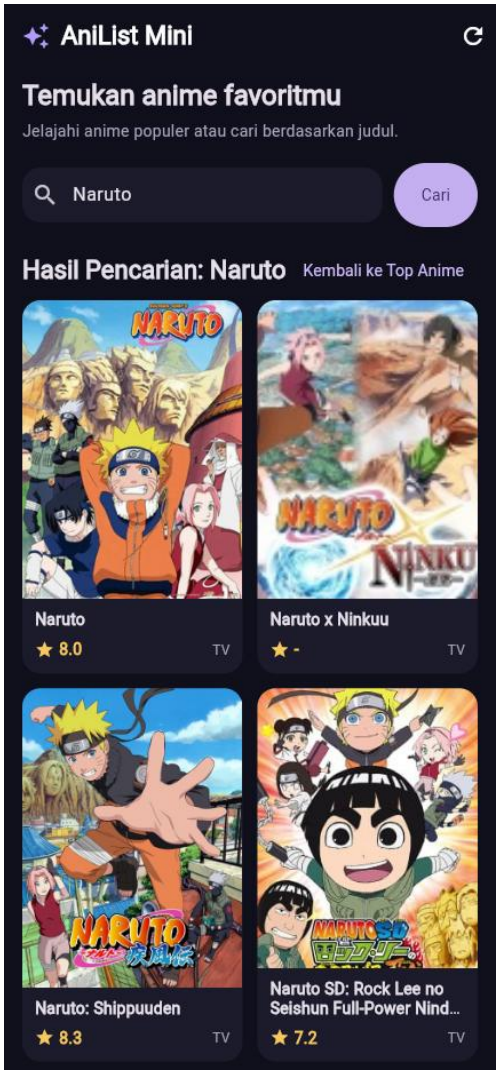
a. Halaman daftar anime

Halaman ini adalah halaman utama. Pada bagian atas terdapat kolom pencarian, dan di bawahnya daftar anime. Setiap baris menampilkan poster, judul anime, skor rating bintang, dan tipe tayangan. Mengetuk salah satu baris membuka halaman detail.



Gambar 2.1 Halaman Daftar Anime

Keterangan: Daftar 24 anime terpopuler beserta kolom pencarian pada bagian atas.

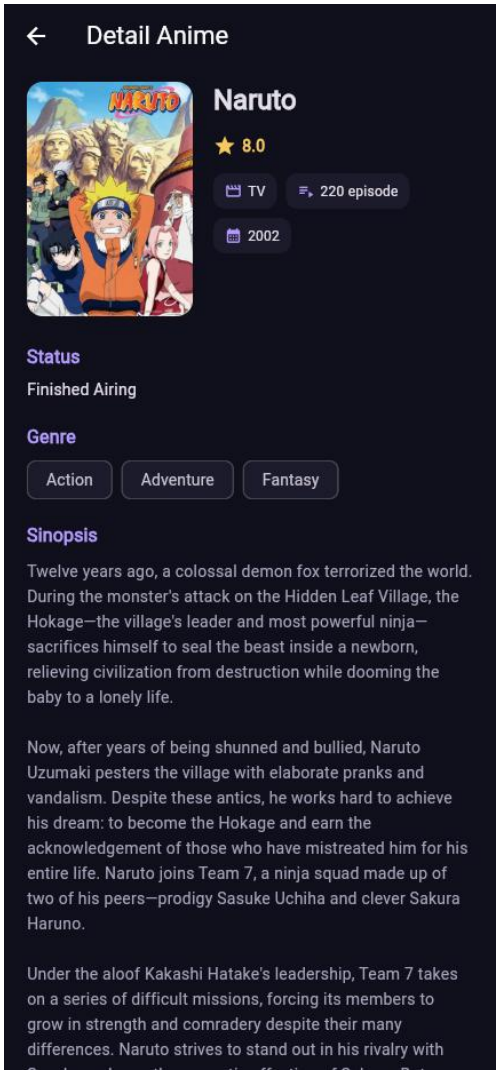


Gambar 2.2 Fitur Pencarian Anime

Keterangan: Daftar tersaring sesuai kata kunci yang diketik pengguna.

b. Halaman detail anime

Bagian atas halaman detail berisi informasi anime (poster besar, judul, rating bintang, tipe tayangan, jumlah episode, tahun rilis, dan status). Di bawahnya terdapat kartu daftar genre dan sinopsis cerita yang dapat digulir.

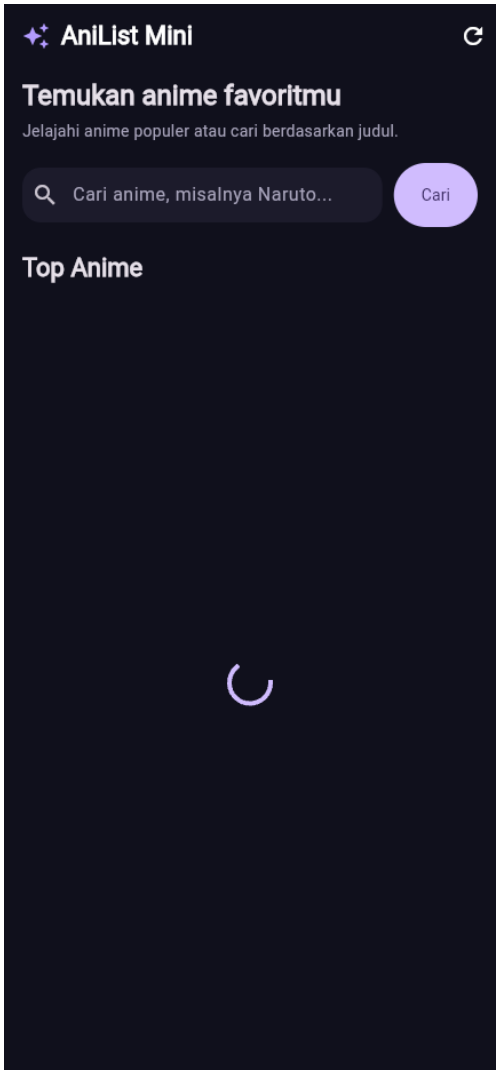


Gambar 2.3 Halaman Detail Anime

Keterangan: Kartu informasi anime dan sinopsis beserta metadatanya.

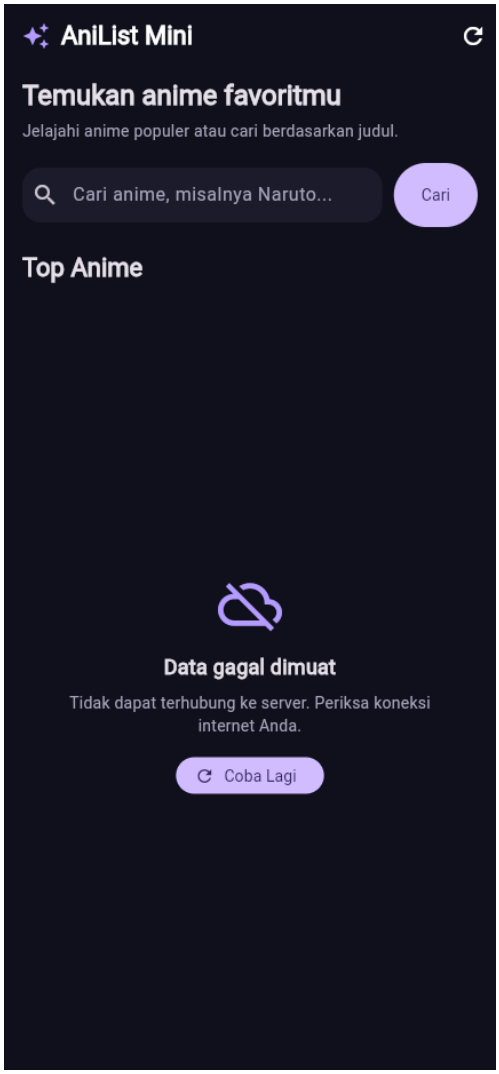
c. Tampilan loading dan error

Saat data sedang diambil, aplikasi menampilkan indikator berputar disertai teks. Jika pengambilan data gagal, aplikasi menampilkan ikon, pesan kesalahan, dan tombol 'Coba Lagi'.



Gambar 2.4 Tampilan Kondisi Loading

Keterangan: Indikator ditampilkan selama aplikasi menunggu respons dari server.



Gambar 2.5 Tampilan Kondisi Error

Keterangan: Pesan kesalahan dan tombol 'Coba Lagi' ditampilkan ketika permintaan gagal.

2.3 Implementasi Pengambilan dan Pengolahan Data dari API

a. URL API dan jenis data

Aplikasi menggunakan API publik Tenrai versi 1 (kompatibel skema Jikan API v4) dengan tiga endpoint berikut.

Endpoint	Metode	Data yang Diberikan
https://api.tenrai.org/v1/top/anime?limit=24	GET	Daftar 24 anime terpopuler beserta informasi ringkasnya.
https://api.tenrai.org/v1/anime?q={query}&limit=24	GET	Daftar anime hasil pencarian berdasarkan kata kunci query.
https://api.tenrai.org/v1/anime/<id>	GET	Detail satu anime spesifik beserta seluruh metadatanya.

b. Struktur respons API

Respons endpoint berbentuk objek JSON dengan daftar anime berada di dalam field 'data':


```
{
  "data": [
    {
      "mal_id": 5114,
      "title": "Fullmetal Alchemist: Brotherhood",
      "images": {
        "jpg": {
          "image_url": "https://cdn.myanimelist.net/images/anime/1208/94745.jpg"
        }
      },
      "score": 9.1,
      "episodes": 64,
      "status": "Finished Airing",
      "type": "TV",
      "genres": [{"name": "Action"}, {"name": "Fantasy"}],
      "synopsis": "After a horrific alchemy experiment goes wrong in the Elric household...",
      "year": 2009
    }
  ]
}
```


Atribut yang dipakai aplikasi dari setiap anime yaitu:

Atribut	Tipe	Keterangan
mal_id	int	Nomor urut/identitas unik anime pada basis data MyAnimeList.
title	String	Judul anime dalam huruf Latin.
images.jpg.image_url	String	URL berkas poster visual anime.
score	double	Nilai evaluasi rating rata-rata pengguna.
episodes	int	Jumlah total episode penayangan anime.
status	String	Status rilis penayangan (Finished / Airing).
genres	List<String>	Daftar kategori genre anime.
synopsis	String	Penjelasan ringkas sinopsis alur cerita anime.

c. Proses pengiriman permintaan HTTP

Pengiriman permintaan dilakukan pada class AnimeService. Alamat dasar API disimpan sebagai konstanta, lalu permintaan GET dikirim menggunakan http.Client dengan batas waktu 20 detik. Diterapkan pembatasan jeda permintaan (_waitForRequestSlot) sebesar 400 ms agar terhindar dari rate limit. Setelah respons diterima, status HTTP diperiksa: jika bukan 200, aplikasi melempar AnimeException berisi pesan kesalahan. Jika berhasil, isi respons didekode dengan jsonDecode dan field data diambil.

```
Future<http.Response> _get(Uri uri) async {
  try {
    await _waitForRequestSlot();
    final response = await _client.get(uri).timeout(const Duration(seconds: 20));
    if (response.statusCode < 200 || response.statusCode >= 300) {
      final msg = response.statusCode == 429
          ? 'Terlalu banyak permintaan ke server. Tunggu sebentar, lalu coba lagi.'
          : 'Server mengembalikan kesalahan (HTTP ${response.statusCode}).';
      throw AnimeException(msg);
    }
    return response;
  } on AnimeException {
    rethrow;
  } on TimeoutException {
    throw const AnimeException('Permintaan terlalu lama. Periksa koneksi internet Anda.');
  } on http.ClientException {
    throw const AnimeException('Tidak dapat terhubung ke server. Periksa koneksi internet Anda.');
  } catch (_) {
    throw const AnimeException('Terjadi kesalahan saat mengambil data anime.');
  }
}
```

Potongan Kode 2.1 Fungsi _get pada anime_service.dart

d. Pengolahan JSON menjadi objek Dart

Setiap objek JSON diubah menjadi objek AnimeModel menggunakan factory constructor AnimeModel.fromJson. Nilai angka dikonversi dengan '(json[...] as num?)?.toInt()', sedangkan nilai teks yang mungkin kosong diberi nilai default melalui operator '??'.

```dart
factory AnimeModel.fromJson(Map<String, dynamic> json) {
  final images = json['images'] as Map<String, dynamic>?;
  final jpg = images?['jpg'] as Map<String, dynamic>?;
  final genresJson = json['genres'] as List<dynamic>? ?? [];

  return AnimeModel(
    id: (json['mal_id'] as num?)?.toInt() ?? 0,
    title: json['title'] as String? ?? 'Judul tidak tersedia',
    imageUrl: jpg?['image_url'] as String? ?? '',
    score: (json['score'] as num?)?.toDouble() ?? 0,
    episodes: (json['episodes'] as num?)?.toInt(),
    synopsis: json['synopsis'] as String? ?? 'Sinopsis belum tersedia.',
    status: json['status'] as String? ?? 'Status tidak tersedia',
    genres: genresJson
        .map((genre) => (genre as Map<String, dynamic>)['name'] as String?)
        .whereType<String>()
        .toList(),
    type: json['type'] as String? ?? 'Tipe tidak tersedia',
    year: (json['year'] as num?)?.toInt(),
  );
}
```

Potongan Kode 2.2 Factory AnimeModel.fromJson pada anime_model.dart

```dart
Future<List<AnimeModel>> fetchTopAnime() =>
    _fetchList(Uri.parse('$_baseUrl/top/anime?limit=24'));
```

Potongan Kode 2.3 Fungsi fetchTopAnime pada anime_service.dart

```dart
Future<List<AnimeModel>> searchAnime(String query) {
  final trimmed = query.trim();
  if (trimmed.isEmpty) return fetchTopAnime();
  final uri = Uri.parse('$_baseUrl/anime')
      .replace(queryParameters: {'q': trimmed, 'limit': '24'});
  return _fetchList(uri);
}
```

Potongan Kode 2.4 Fungsi searchAnime pada anime_service.dart

e. Implementasi asynchronous programming

Pengambilan data membutuhkan waktu karena aplikasi berkomunikasi dengan server, sehingga fungsi dibuat asinkron dengan kata kunci async dan await dan mengembalikan Future.

Pada halaman daftar, Future disimpan pada variabel _animeFuture di dalam initState. Dengan cara ini permintaan hanya dikirim satu kali saat halaman dibuat, dan tidak diulang setiap kali widget dibangun ulang.

```
@override
void initState() {
  super.initState();
  _service = widget.service ?? AnimeService();
  _animeFuture = _service.fetchTopAnime();
}
```

Potongan Kode 2.5 Inisialisasi Future pada initState

f. Penanganan kondisi loading, success, dan error

Tampilan halaman disesuaikan dengan status Future menggunakan FutureBuilder.

Kondisi	Pemicu	Tampilan
Loading	connectionState belum done (menunggu server).	Indikator CircularProgressIndicator di tengah layar.
Error	snapshot.hasError bernilai true (permintaan gagal).	Ikon, pesan kesalahan, dan tombol 'Coba Lagi' yang mengulangi permintaan.
Empty	snapshot.hasData true dan list kosong.	Teks 'Anime tidak ditemukan' dan tombol kembali ke Top Anime.
Success	Future selesai dan snapshot.data tersedia.	Kolom pencarian dan katalog anime dalam bentuk kisi (grid).

```dart
FutureBuilder<List<AnimeModel>>(
  future: _animeFuture,
  builder: (context, snapshot) {
    // Kondisi 1: Loading
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }
    // Kondisi 2: Error
    if (snapshot.hasError) {
      return _MessageState(
        icon: Icons.cloud_off, title: 'Data gagal dimuat',
        message: '${snapshot.error}', buttonText: 'Coba Lagi',
        onPressed: _retry,
      );
    }
    // Kondisi 3: Success
    final animeList = snapshot.data ?? [];
    if (animeList.isEmpty) {
      return _MessageState(
        icon: Icons.search_off, title: 'Anime tidak ditemukan',
        buttonText: 'Tampilkan Top Anime', onPressed: _showTopAnime,
      );
    }
    return GridView.builder( ... );
  },
)
```

Potongan Kode 2.6 FutureBuilder pada home_page.dart

Jenis kesalahan yang ditangani pada AnimeService beserta pesan yang ditampilkan kepada pengguna:

Jenis Kesalahan	Pesan untuk Pengguna
Tidak ada internet (SocketException / ClientException)	Tidak dapat terhubung ke server. Periksa koneksi internet Anda.
Waktu habis (TimeoutException, 20 detik)	Permintaan terlalu lama. Periksa koneksi internet Anda.
Rate limit (HTTP status 429)	Terlalu banyak permintaan ke server. Tunggu sebentar, lalu coba lagi.
Status HTTP bukan 200	Server mengembalikan kesalahan (HTTP {kode}).
JSON tidak valid (FormatException)	Data dari server tidak dapat dibaca.
Struktur atau tipe data tidak sesuai	Format data dari server tidak sesuai.

g. Penyajian data pada antarmuka

Ketika data berhasil diperoleh, daftar ditampilkan dengan GridView.builder yang hanya membangun item yang terlihat di layar sehingga efisien. Fitur pencarian bekerja dengan menyaring kata kunci tanpa permintaan baru ke server pada setiap huruf:

```dart
void _search() {
  FocusScope.of(context).unfocus();
  final query = _searchController.text.trim();
  setState(() {
    _sectionTitle = query.isEmpty ? 'Top Anime' : 'Hasil Pencarian: $query';
    _animeFuture = query.isEmpty
        ? _service.fetchTopAnime()
        : _service.searchAnime(query);
  });
}
```

Potongan Kode 2.7 Fungsi _search pada home_page.dart

Halaman detail bekerja dengan pola yang sama: DetailPage menerima data anime awal kemudian FutureBuilder memuat informasi detail anime secara lengkap.

2.4 Pengujian Aplikasi

Pengujian dilakukan dengan dua cara: pengujian otomatis menggunakan flutter test (tanpa memerlukan internet karena memakai MockClient) dan pengujian manual pada emulator atau peramban.

a. Pengujian otomatis

Perintah yang dijalankan: flutter test.

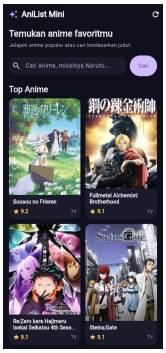
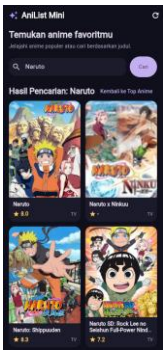
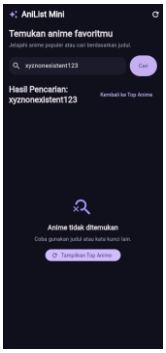
No	Pengujian	Hasil yang Diharapkan	Hasil
1	AnimeModel.fromJson	JSON berubah menjadi objek AnimeModel dengan benar.	Lulus
2	AnimeModel field null	Nilai default aman diberikan saat field opsional null.	Lulus
3	fetchTopAnime sukses	Mengembalikan list AnimeModel sesuai jumlah data.	Lulus
4	searchAnime encoding	Query URL terkodekan dan menerima hasil kosong.	Lulus
5	fetchAnimeDetail sukses	Mengembalikan objek AnimeModel detail tunggal.	Lulus
6	Respons status 429	Melempar AnimeException dengan pesan rate limit.	Lulus
7	Respons status 500	Melempar AnimeException dengan status server.	Lulus
8	Respons bukan JSON	Melempar AnimeException.	Lulus
9	Kegagalan jaringan	Melempar AnimeException pesan koneksi internet.	Lulus
10	Widget: loading & sukses	Indikator loading tampil, lalu kartu anime tampil.	Lulus

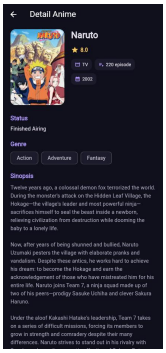
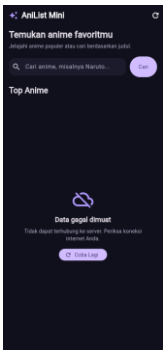


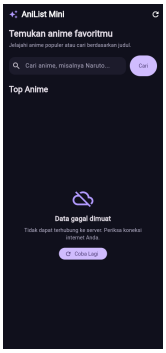
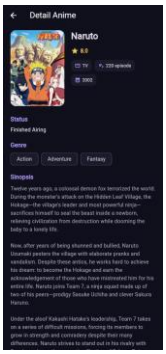
Gambar 2.6 Hasil Perintah flutter test

Keterangan: Seluruh pengujian otomatis beserta statusnya.

b. Pengujian manual

No	Skenario	Langkah	Hasil yang diharapkan	Hasil aktual
1	Memuat daftar anime	Buka aplikasi dengan internet aktif.	Loading tampil, lalu 24 anime terpopuler tampil.	
2	Pencarian ditemukan	Ketik "Naruto" pada kolom pencarian.	Hanya judul terkait Naruto yang tampil.	
3	Pencarian tidak ditemukan	Ketik kata yang tidak ada, misalnya "xyz".	Teks "Anime tidak ditemukan" tampil.	

No	Skenario	Langkah	Hasil yang diharapkan	Hasil aktual
4	Membuka detail anime	Ketuk salah satu anime.	Loading, lalu rincian dan sinopsis tampil.	
5	Tanpa internet (daftar)	Matikan internet, lalu buka aplikasi.	Pesan error dan tombol "Coba Lagi" tampil.	
6	Coba lagi	Nyalakan internet, tekan "Coba Lagi".	Loading, lalu daftar anime tampil.	

No	Skenario	Langkah	Hasil yang diharapkan	Hasil aktual
7	Tanpa internet (detail)	Matikan internet, ketuk anime.	Pesan error pada halaman detail tampil.	
8	Deskripsi anime	Buka detail anime pada halaman detail.	Sinopsis dan genre tampil rapi.	

BAB III. PENUTUP

3.1 Kesimpulan

Aplikasi AniList Mini berhasil dikembangkan menggunakan Flutter dan Dart. Aplikasi mengambil data secara dinamis dari API Tenrai melalui permintaan HTTP, memeriksa status respons, mengubah JSON menjadi objek Dart dengan class model AnimeModel, lalu menyajikannya pada dua halaman, yaitu daftar anime dan detail anime.

Asynchronous programming diterapkan menggunakan Future, async/await, dan FutureBuilder sehingga aplikasi dapat menampilkan kondisi loading, success, dan error dengan jelas. Kesalahan seperti tidak ada internet, waktu habis, status server yang salah, dan data yang tidak valid ditangani dengan pesan yang mudah dipahami serta tombol untuk

Berdasarkan hasil pengujian otomatis dan manual pada Bab 2.4, seluruh skenario berjalan sesuai harapan sehingga aplikasi dinyatakan telah berfungsi sesuai dengan tujuan pengembangannya.

3.2 Saran

Beberapa saran untuk pengembangan pada tahap berikutnya adalah sebagai berikut.

- Menambahkan penyimpanan lokal (cache) agar daftar anime dan detail yang sudah dibuka tetap dapat dibaca tanpa internet.
- Menambahkan fitur penanda (bookmark) dan riwayat terakhir dibuka.
- Menerapkan pagination agar aplikasi dapat memuat lebih banyak judul anime.
- Menambahkan filter genre dan penyortiran berdasarkan rating atau tahun rilis.
- Menerapkan state management seperti Provider atau Riverpod bila fitur bertambah banyak.

DAFTAR PUSTAKA

Flutter Team. (2026). Flutter documentation. https://docs.flutter.dev (diakses 20 Oktober 2026).

Flutter Team. (2026). Fetch data from the internet. https://docs.flutter.dev/cookbook/networking/fetch-data (diakses 20 Oktober 2026).

Flutter Team. (2026). FutureBuilder class. https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html (diakses 20 Oktober 2026).

Dart Team. (2026). Dart programming language documentation. https://dart.dev/guides (diakses 20 Oktober 2026).

Dart Team. (2026). http package. https://pub.dev/packages/http (diakses 20 Oktober 2026).

Flutter Team. (2026). flutter_lints package. https://pub.dev/packages/flutter_lints (diakses 20 Oktober 2026).

Tenrai. (2026). Tenrai API documentation. https://api.tenrai.org (diakses 20 Oktober 2026).